Phase 1 - Grade 2 - NumPy Arrays

Key Learning Points

A Python list can hold anything, so Python type-checks every element every time it does math -- this flexibility costs speed. NumPy's **ndarray** requires every element to be the same type, stored in one contiguous block of memory. Because the data is uniform, math on the WHOLE array happens in one optimized operation instead of looping element-by-element -- this is **vectorization**, the core reason NumPy exists.

Analogy that made it click: a Python list is like a Java ArrayList<Object> (flexible, boxed, overhead per element). A NumPy array is closer to a raw Java primitive array, e.g. int[] -- fixed type, contiguous memory, fast bulk math.

shape
: Attribute giving an array's dimensions as a tuple. 1-D array of 3 elements -> (3,). A 2-row, 3-column 2-D array -> (2, 3) -- rows first, then columns.

Slicing (1-D)
: array[start:stop] -- gets index start up to (not including) stop.

2-D indexing
: array[row, col]. ':' alone means 'every value along that axis' -- e.g. grid[:, 0] means all rows, column 0 only.

Boolean masking
: Filtering using a True/False array as an index: temps[temps > 32]. The vectorized replacement for a list comprehension with an if-filter.

Aggregate functions
: .sum(), .mean(), .max(), .min() -- reduce an array to a single summary value.

Why NumPy is faster (the mechanism)
: Same-type, contiguous memory means no per-element type-checking, and lets NumPy hand the whole array to optimized, pre-compiled C code for the operation in one shot -- not just that it has convenient built-in methods.

Clarified Doubts

My doubt: For grid[0,1] etc., does 'column 1' / 'row 2' mean the 1st/2nd column by counting, or the index?
: Clarification: It means INDEX, not counting position. NumPy is zero-indexed, same as Python lists / Java arrays. 'Column 1' = grid[:, 1] (the index-1 column, which is visually the 2nd column). 'Row 2' = grid[2, :] (the index-2 row, visually the 3rd row).

Examples

```
import numpy as np
prices = np.array([100, 200, 300])
```

```
prices.shape                # (3,)
tax_included = prices * 1.1  # [110. 220. 330.]  -- vectorized, no loop

temps = np.array([30, 32, 35, 28, 40])
cel_to_fahr = temps * 9/5 + 32   # [ 86.  89.6  95.  82.4 104. ]
temps[1:3]        # [32 35]

grid = np.array([[1, 3], [4, 5], [8, 5]])
grid.shape        # (3, 2)
grid[0, 1]        # 3
grid[:, 1]        # [3 5 5]
grid[2, :]        # [8 5]

mask = temps > 32
hot_days = temps[mask]          # [35 40]
hot_days = temps[temps > 32]    # same, one line

temps.sum(); temps.mean(); temps.max(); temps.min()
```

Quiz Q&A;

Q1. In plain English, why is a NumPy array faster than a Python list for math operations?

A: Same-type, contiguous memory means no per-element type-checking, and enables vectorized (whole-array, no-loop) operations running in optimized C code, instead of a Python-level loop.

Q2. Given arr = np.array([5, 10, 15, 20, 25]), get only values ≤ 15.

A: arr[arr <= 15] -> [5 10 15]

Q3. Given m = np.array([[2,4,6],[8,10,12]]): (a) m.shape? (b) code to get all rows, only the last column.

A: (a) (2, 3). (b) m[:, -1] or m[:, 2] -> [6 12]

Q4. What is the difference between arr[1:3] and arr[1, 3]?

A: arr[1:3] is SLICING -- valid on a 1-D array, returns a sub-range. arr[1, 3] is 2-D (row,col) indexing -- only valid on an array with two axes; on a 1-D array it raises IndexError: too many indices for array.

Q5. True or False: a NumPy array can hold a mix of integers and strings, just like a Python list.

A: False -- a NumPy array requires all elements to be the same type, stored in one contiguous memory block. Mixed types defeat the entire design.